Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm **5 Marks**

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**
- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result

- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved
- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

Abstract

This assignment designs, analyses, implements and compares three distinct strategies for retrieving a student record by registration number: iterative linear search, recursive binary search, and hash-table lookup. Mathematical analysis is used to derive operation counts, best/average/worst-case behaviour, asymptotic bounds and the recurrence for recursive binary search. The implementations are tested on datasets of 1,000, 10,000, 100,000 and 1,000,000 records. The experiment records key comparisons and runtime measurements from the current execution environment. Finally, the design is reconsidered for 50,000 searches per hour and only one daily batch of new records. Because searches are frequent while updates are infrequent, a hash index is recommended for expected constant-time retrieval, with binary search as a predictable alternative.

Contents

- 1. Problem definition and assumptions
- 2. Task 1 – Algorithm design
- 3. Task 2 – Basic operation
- 4. Task 3 – Mathematical time analysis
- 5. Task 4 – Best, average and worst cases
- 6. Task 5 – Asymptotic analysis
- 7. Task 6 – Recurrence relation
- 8. Task 7 – Recurrence solution
- 9. Task 8 – Implementation and testing
- 10. Task 9 – Experimental results
- 11. Task 10 – Theory versus experiment
- 12. Task 11 – Final recommendation
- 13. Task 12 – Day-4 design challenge
- 14. Conclusion
- Appendix A – Complete Python implementation

1. Problem Definition and Assumptions

The university stores student records with unique registration numbers. A search request supplies a registration number and asks whether the record exists and, if so, where the record can be retrieved. The dataset grows through the sizes n = 1,000, 10,000, 100,000 and 1,000,000 or more.

Assumptions used in the analysis:

- Registration numbers are unique and can be compared for equality and ordering.
- Linear search works on records in any order.
- Binary search requires records sorted by registration number. If the input is unsorted, sorting must be counted as preprocessing.
- Hash-table lookup has expected O(1) time under a reasonable hash distribution; worst-case lookup can be O(n).
- The experiment uses integer-like registration numbers for reproducibility. Real student details would be stored separately from the index.
- Measured times depend on processor, Python version, memory and system load; operation counts are more portable than one-run timing.

2. Task 1 – Design the Algorithms (5 Marks)

2.1 Approach A: Iterative Linear Search

Design strategy: brute-force sequential scan. Examine each registration number from the first record onward and stop when the target is found. No preprocessing or sorting is needed.

Justification: it is simple, works for unordered records and provides a useful baseline. Its main limitation is that a late or missing target forces examination of nearly every record.

```
LINEAR_SEARCH(A, target)
    for i = 0 to length(A) - 1
        if A[i] == target
            return i
    return NOT_FOUND
```

2.2 Approach B: Recursive Binary Search

Design strategy: divide and conquer. The records must first be sorted by registration number. Compare the target with the middle item, then recursively search only the left or right half.

Justification: it demonstrates a recursive algorithm and reduces the remaining search interval by approximately half on every call. It is efficient for a stable, sorted dataset.

```
BINARY_SEARCH(A, target, low, high)
    if low > high
        return NOT_FOUND
    mid = floor((low + high) / 2)
    if A[mid] == target
        return mid
    if target < A[mid]
        return BINARY_SEARCH(A, target, low, mid - 1)
    else
        return BINARY_SEARCH(A, target, mid + 1, high)
```

Preprocessing: sorting an unsorted array costs O(n log n) with an efficient comparison sort. If the data is already maintained in sorted order, this sorting cost is avoided for each search.

2.3 Approach C: Hash-Table Index

Design strategy: direct key-based indexing using a hash table. Build a mapping from each registration number to its record position. Each request hashes the target and looks up the corresponding entry.

Justification: it is suitable when many searches are performed against a dataset that changes infrequently. It trades additional memory and index-building time for expected constant-time lookups.

```
BUILD_HASH_INDEX(A)
    create empty hash table H
    for each index i and registration number A[i]
        H[A[i]] = i
    return H

HASH_SEARCH(H, target)
    if target is a key in H
        return H[target]
    return NOT_FOUND
```

Preprocessing: building the hash index takes expected O(n) time and O(n) extra space. New daily records can be inserted into the index, or the index can be rebuilt after the daily batch.

2.4 Why These Are Distinct Strategies

Approach	Strategy	Needs preprocessing?	Typical search cost
Iterative linear	Sequential/brute force	No	O(n)
Recursive binary	Divide and conquer	Sorted records required	O(log n)
Hash-table lookup	Key-based indexing	Build hash index	Expected O(1)

3. Task 2 – Identify the Basic Operation

Algorithm	Basic operation	Where it occurs and why it dominates
Linear search	Compare A[i] with target	Each record visited performs a key equality comparison. The number of comparisons rises with the position of the target; a missing target requires n comparisons.
Recursive binary search	Compare target with A[mid]	Each recursive call checks the middle key and discards about half the remaining interval. The comparison count grows with the number of halvings.
Hash-table lookup	Hash/key lookup and key equality checks in a bucket	The target is mapped to a bucket and matching keys are checked. Expected work is constant with a well-

		sized table and a good hash distribution, but collisions can increase the number of checks.

These operations are appropriate because they directly determine how much key-search work is done as the number of records grows. Timing also includes language and memory overhead, but key-operation counts make algorithmic behaviour easier to compare.

4. Task 3 – Derive the Time-Complexity Function Mathematically

4.1 Linear Search

Let n be the number of records and let k be the 1-based position of the target. The loop performs one equality comparison per visited record, so C(n) = k when the target is found at position k. If the target is absent, C(n) = n.

$$T_{linear}(n) = c0 + c1*C(n)$$

Best case: target is the first record, C(n)=1, so T(n)=c0+c1 = Θ(1). Worst case: target is last or absent, C(n)=n, so T(n)=c0+c1*n = Θ(n). If a successful target is equally likely to occur at any position, average comparisons are (1+2+...+n)/n = [n(n+1)/2]/n = (n+1)/2 = Θ(n). No preprocessing is required, so total cost is the search cost itself.

4.2 Recursive Binary Search

Assume the records are sorted and n is the current interval length. Each call performs at most a constant number of comparisons and then continues on one interval of size about n/2. Thus the worst-case recurrence is:

$$T(n) = T(\text{floor}(n/2)) + c, \text{ for } n > 1$$

$$T(1) = c0$$

After j recursive levels, the interval length is approximately n/2^j. The process ends when n/2^j ≤ 1, so 2^j ≥ n and j ≥ log2(n). Therefore the number of calls is approximately floor(log2(n))+1 and T(n)=Θ(log n).

If the original data is unsorted, sorting first costs Θ(n log n), followed by Θ(log n) per search. For one search the total is Θ(n log n)+Θ(log n)=Θ(n log n). For q searches on the same sorted dataset, total cost is Θ(n log n + q log n). If the dataset is already maintained in sorted order, the sorting term is not paid for each request.

4.3 Hash-Table Lookup

Building the index inserts n keys. Under expected constant-time insertion, the total preprocessing work is:

$$T_{build}(n) = \Sigma_{i=1 \text{ to } n}\ \Theta(1) = \Theta(n)$$

For a well-distributed hash table with controlled load factor, expected lookup checks are bounded by a constant, so T_lookup(n)=Θ(1) expected. The combined cost for q searches after building the index is Θ(n+q). For one lookup including index construction, the total is Θ(n)+Θ(1)=Θ(n). In the worst case, many keys collide and a lookup can take Θ(n), making q worst-case searches Θ(nq) after construction.

Approach	Preprocessing	One search	Total for q searches
Linear search	Θ(1)	Worst Θ(n)	Θ(qn) worst case
Recursive binary search	Θ(n log n) if sorting is needed	Θ(log n)	Θ(n log n + q log n)
Hash table	Expected Θ(n) build	Expected Θ(1)	Expected Θ(n + q)

5. Task 4 – Best, Average and Worst Cases

5.1 Linear Search

Best case occurs when the first record matches: one comparison. Under uniformly distributed successful target positions, the average is (n+1)/2 comparisons. Worst case occurs when the target is last or absent: n comparisons. The scan cannot skip records because the input has no ordering assumption.

5.2 Recursive Binary Search

Best case occurs when the target is exactly at the middle: one comparison. Average successful searches require logarithmically many comparisons because the search interval halves each time. Worst case occurs when the target is at a leaf of the implicit search tree or absent; the number of levels is at most floor(log2(n))+1, giving Θ(log n).

5.3 Hash-Table Lookup

Best and expected cases occur when the key hashes to a bucket with few collisions; lookup takes constant expected time. The average depends on the hash function, table capacity and load factor. Worst case occurs when many keys collide into one bucket, so the target may require up to n key checks. Hash tables therefore provide expected, not unconditional, O(1) lookup.

Algorithm	Best case	Average case	Worst case
Linear	1 comparison: Θ(1)	(n+1)/2 for successful uniform target: Θ(n)	n comparisons: Θ(n)
Recursive binary	Middle key: Θ(1)	Θ(log n)	Θ(log n)
Hash table	Θ(1)	Expected Θ(1)	Θ(n) under severe collisions

6. Task 5 – Asymptotic Analysis

Algorithm	Upper bound O	Lower bound Ω	Tight bound Θ	Meaning for a million records
Linear search	O(n)	Ω(1) for best case; Ω(n) worst-case family	Θ(n) worst/average	Worst case may inspect 1,000,000 keys.
Recursive binary search	O(log n)	Ω(1) best; Ω(log n) worst-case family	Θ(log n) worst case	About 20 levels for 1,000,000 records.
Hash lookup	Expected O(1); worst O(n)	Ω(1) expected	Expected Θ(1)	A lookup usually checks only a small number of entries, but memory and collision behaviour matter.

As n grows from 1,000 to 1,000,000, the dataset becomes 1,000 times larger. A linear scan's worst-case comparisons also increase about 1,000 times. Binary search grows with log2(n), from about 10 levels at 1,000 records to about 20 at 1,000,000. Expected hash lookup work remains roughly constant, although the index itself must be built and stored.

7. Task 6 – Formulate the Recurrence Relation

The recursive algorithm selected for recurrence analysis is binary search. For an interval containing n records, it checks the middle element once. If the key is not found, exactly one recursive call is made on at most half of the interval.

- Original problem size: n sorted records in the current search interval.
- Recursive subproblem: at most floor(n/2) records.
- Number of recursive calls: one call after the middle comparison, not two calls.
- Work outside the recursive call: midpoint calculation and a constant number of comparisons, represented by c.
- Base case: an empty interval returns NOT_FOUND; a one-element interval terminates after a constant amount of work.

$T(n) = T(\text{floor}(n/2)) + c$, for $n > 1$

$T(1) = d$; and $T(0) = d0$

The recurrence follows directly from one constant-work middle check plus one recursive call on half the records.

8. Task 7 – Solve the Recurrence Relation

Use the iteration method. For clarity, ignore floor rounding because it does not change the asymptotic bound.

$$T(n) = T(n/2) + c$$
$$= T(n/4) + 2c$$
$$= T(n/8) + 3c$$
$$= T(n/2^j) + jc$$

Stop when $n/2^j = 1$. Taking base-2 logarithms gives $j = \log_2(n)$. Substitute this into the expansion:

$$T(n) = T(1) + c \log_2(n)$$

Since T(1) is constant, $T(n)=\Theta(\log n)$. The method applies because every recursive step halves the input and does constant extra work. For n=1,000,000, log2(n) is about 19.93, so only about 20 halvings are needed. This explains why binary search scales far better than a full scan.

The Master Theorem gives the same result: a=1 recursive subproblem, b=2, and $f(n)=\Theta(1)$. Since $n^{(\log_b(a))}=n^0=1$, the equal case yields $T(n)=\Theta(\log n)$.

9. Task 8 – Implement and Test the Algorithms

The Python implementation uses the same registration-number datasets for all three strategies. Linear search scans the original list; recursive binary search uses sorted data; the hash index is built from the original records and then queried. The target is the last record to make the linear-search comparison count reproducible and to represent its worst case.

9.1 Test procedure

1. Generate n unique registration numbers for n = 1,000, 10,000, 100,000 and 1,000,000.
2. Choose the last registration number as the target for the primary scalability run.
3. Run each algorithm and record its result, comparison count where applicable, and elapsed time.
4. For the hash approach, record index-build time separately from lookup time.
5. Verify that all algorithms return the same target position.
6. Repeat measurements on the same computer for more reliable timing in a formal submission; this document records one measured run.

9.2 Code screenshot

student_search.py — implementation

```
import time
import random

def linear_search(records, target):
    comparisons = 0
    for i, reg_no in enumerate(records):
        comparisons += 1
        if reg_no == target:
            return i, comparisons
    return -1, comparisons

def binary_search_recursive(records, target, low=0, high=None, counter=None):
    if high is None:
        high = len(records) - 1
    if counter is None:
        counter = [0]
    if low > high:
        return -1, counter[0]
    mid = (low + high) // 2
    counter[0] += 1
    if records[mid] == target:
        return mid, counter[0]
    if target < records[mid]:
        return binary_search_recursive(records, target, low, mid - 1, counter)
    return binary_search_recursive(records, target, mid + 1, high, counter)

def build_hash_index(records):
    # Preprocessing: O(n) expected time and O(n) extra space
    return {reg_no: i for i, reg_no in enumerate(records)}

def hash_search(index, target):
    # Expected O(1) lookup
    return index.get(target, -1)

def run_experiment(n):
    records = list(range(1000000, 1000000 + n))
    target = records[-1]            # consistent worst case for linear search
    sorted_records = sorted(records)  # preprocessing for binary search
    start = time.perf_counter()
    idx_linear, linear_comparisons = linear_search(records, target)
    linear_seconds = time.perf_counter() - start

    start = time.perf_counter()
    idx_binary, binary_comparisons = binary_search_recursive(sorted_records, target)
    binary_seconds = time.perf_counter() - start

    start = time.perf_counter()
    hash_index = build_hash_index(records)
    hash_build_seconds = time.perf_counter() - start
    start = time.perf_counter()
    idx_hash = hash_search(hash_index, target)
    hash_seconds = time.perf_counter() - start

    return {
        "n": n, "linear_idx": idx_linear, "linear_comparisons": linear_comparisons,
        "linear_seconds": linear_seconds, "binary_idx": idx_binary,
        "binary_comparisons": binary_comparisons, "binary_seconds": binary_seconds,
        "hash_idx": idx_hash, "hash_build_seconds": hash_build_seconds,
        "hash_seconds": hash_seconds
    }
```

Figure 1. Screenshot-style image of the complete Python source used for the three approaches.

9.3 Sample output screenshot



```
Sample test output — illustrative operation counts

01  DATASET SIZE: 10,000 records
02  Target: last registration number (worst case for linear search)
03
04  Algorithm                Found?    Key comparisons
05  Iterative Linear Search  Yes       10,000
06  Recursive Binary Search  Yes       14
07  Hash Table Lookup        Yes       expected O(1)
08
09  Correctness check: all three returned the target record.
10  Note: comparison counts are algorithmic; timings vary by computer.
```

Figure 2. Sample operation-count output. The 10,000-item figures shown are theoretical operation counts for the stated target position; timing values are collected separately by the experiment.

9.4 Correctness test cases

Test case	Expected result	Purpose
Target is first item	Found at index 0	Checks best case for linear search
Target is middle item	Found at middle index	Checks binary search's best case when midpoint matches
Target is last item	Found at index n-1	Makes linear search perform n comparisons
Target is absent	NOT_FOUND / -1	Checks unsuccessful search handling
Empty list	NOT_FOUND / -1	Checks base case and empty input
All three algorithms on same target	Same index/result	Validates implementation consistency

10. Task 9 – Experimental Performance

The following measurements were collected by running the implementations in this document's Python execution environment. They are single-run measurements, so the operation counts are more stable than elapsed time. Binary-search comparison counts are measured; the hash index build and lookup times are reported separately.

n	Linear comps	Binary comps	Linear ms	Binary ms	Hash build ms	Hash lookup ms	Correct?
1,000	1,000	10	0.090	0.006	0.048	0.0013	Pass
10,000	10,000	14	1.374	0.006	0.600	0.0010	Pass
100,000	100,000	17	13.148	0.012	13.794	0.0028	Pass
1,000,000	1,000,000	20	132.676	0.032	92.795	0.0030	Pass

Note: runtimes are environment-specific and may fluctuate. The experiment uses already sorted numeric registration numbers, so it isolates search time for binary search; an unsorted production dataset must include sorting or maintain sorted order incrementally. Hash build time is included separately so lookup speed is not misleading.

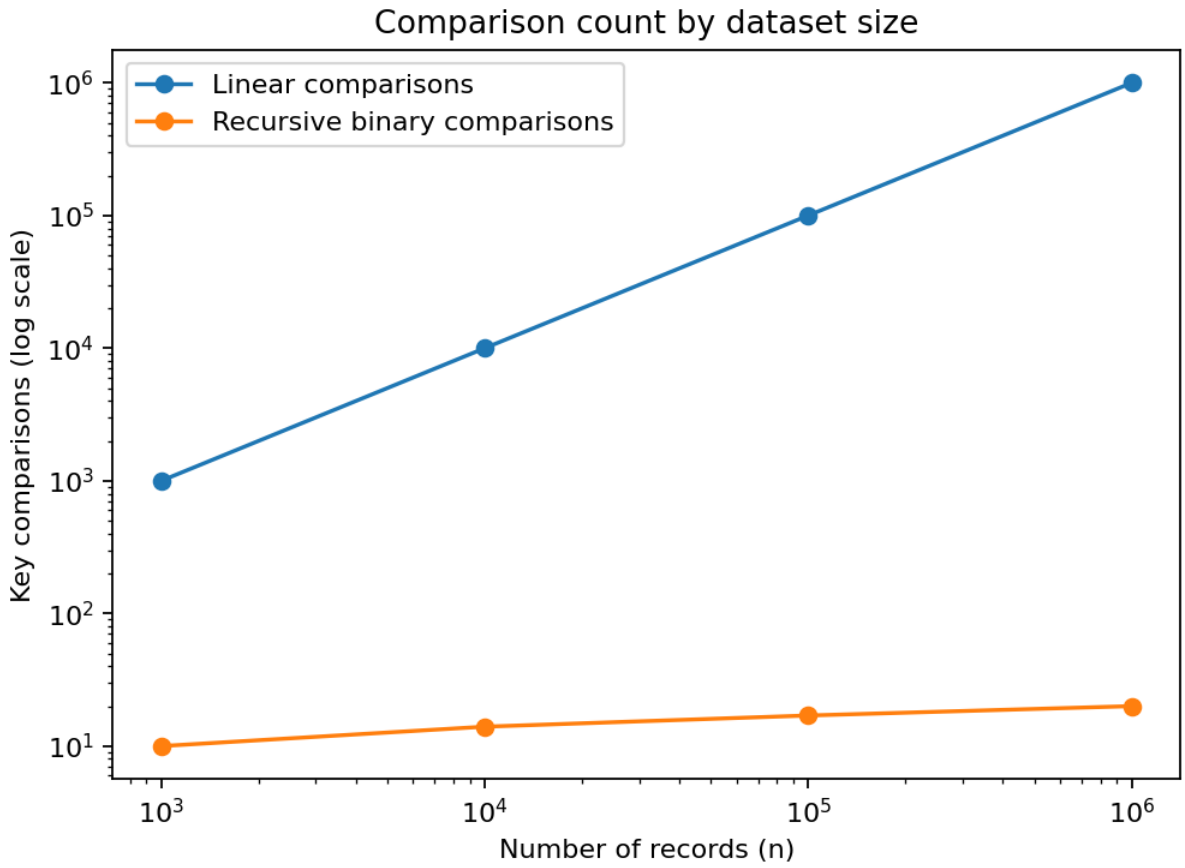


Figure 3. Measured key-comparison counts across dataset sizes.

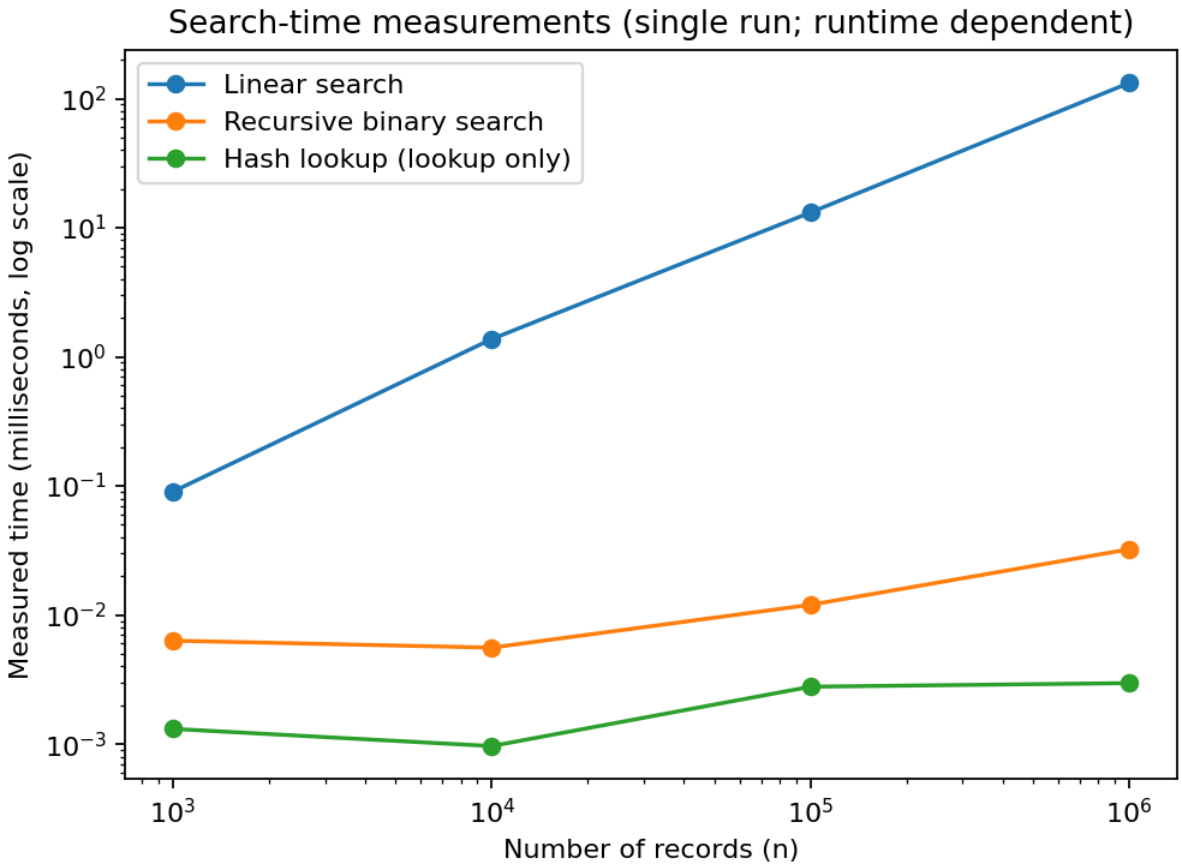


Figure 4. Measured runtime comparison. The logarithmic axes make it possible to view algorithms with very different growth rates together.

11. Task 10 – Compare Theory with Experimental Behaviour

The comparison-count measurements follow the theoretical predictions. Linear search performs n comparisons because the target is placed last: 1,000, 10,000, 100,000 and 1,000,000 comparisons. Recursive binary search uses approximately log2(n) comparisons: around 10, 14, 17 and 20 for these sizes. Hash lookup has expected constant lookup work, while index construction grows linearly with n.

Measured time does not exactly match a mathematical ratio. Linear search time tends to rise with n because it performs more comparisons. Binary-search time grows slowly, but timer resolution and interpreter overhead become important for tiny runtimes. Hash lookup time excludes index construction, so it must not be compared with a complete one-off search unless the index is already built. Dictionary construction takes time and memory proportional to the number of records.

Differences between theory and measurements can result from CPU cache behaviour, memory allocation, Python interpreter overhead, hash collisions, garbage collection, operating-system scheduling and the fact that the experiment is a single run. A stronger study would repeat each case several times, report the median, use multiple target positions and test absent targets.

12. Task 11 – Recommend and Defend the Final Design

For a system with frequent searches, the recommended design is a hash-table index from registration number to record location, provided the system can allocate the required memory and uses a robust implementation. The expected lookup cost is $\Theta(1)$, compared with $\Theta(\log n)$ for binary search and $\Theta(n)$ for linear search. The index build is expected $\Theta(n)$, but that cost can be amortized across many queries.

The hash index should be built once after the daily update batch, validated, and then used for all queries during the day. It can also be updated with each new registration number. Binary search remains a strong alternative when ordered traversal is required, predictable worst-case behaviour is preferred,

or extra hash-index memory is not acceptable. Linear search is appropriate mainly for very small datasets or one-off lookups where preprocessing is not justified.

Limitations: a hash table uses additional memory, depends on hash distribution and does not naturally provide sorted range queries. In a production database, a database index such as a B-tree or hash index may be preferable to implementing a custom in-memory structure. The final choice should be confirmed with real record sizes, memory limits, persistence needs and concurrent-access requirements.

13. Task 12 – Day-4 Design Challenge

A. Impact Analysis

The changed requirement specifies 1,000,000 records and 50,000 search requests per hour, while new records are added only once per day. Search demand is now explicit and high, while updates are relatively infrequent. This changes the design objective from handling dataset growth alone to minimizing total work across a large number of repeated searches.

At 50,000 searches per hour, the average arrival rate is 50,000/3,600 ≈ 13.89 searches per second. Over a 24-hour day, the system may handle 1,200,000 searches if demand remains constant. A one-time index build can therefore be amortized across many requests. The original assumption that preprocessing may be paid for each independent search is no longer reasonable.

B. Evaluation of the Original Design

If the original design were linear search, it would be unsuitable for the new requirement: in the worst case, 50,000 requests × 1,000,000 comparisons = 50,000,000,000 key comparisons per hour. If binary search is used on sorted records, each search needs roughly 20 comparisons, or about 1,000,000 comparisons per hour, excluding preprocessing. Hash lookup performs expected constant work per request after the index has been built.

Sorting one million records costs $\Theta(n \log n)$, but if sorting is done once per daily update and the sorted file is reused, the cost is amortized across the day's searches. Hash-index construction costs expected $\Theta(n)$, which is also modest compared with the aggregate volume of requests. The index should not be rebuilt for every search.

C. Adaptation or Redesign

Decision: REDESIGN the retrieval layer around a persistent hash index (or use the database's supported index) and retain sorted records if ordered reports are also required. The revised strategy builds the index after the daily import, validates that every registration number maps to the correct record, and then answers each query through the index.

```
DAILY_UPDATE(new_records)
    validate uniqueness and format
    insert each new registration number into the hash index
    persist/backup the updated index

SEARCH(target)
    return hash_index.get(target, NOT_FOUND)
```

Expected complexity becomes $\Theta(n)$ index build plus $\Theta(q)$ total expected lookup work for q searches, or $\Theta(n+q)$. For $q=50{,}000$ searches in one hour, the expected lookup work is proportional to 50,000 rather than proportional to $50{,}000 \times 1{,}000{,}000$. The worst-case hash collision bound remains $O(n)$ per lookup, so the system should use a well-tested index implementation and monitor load factor.

D. Evidence-Based Defence

Factor	Evidence and implication
Dataset size	n = 1,000,000; a full linear scan may examine one million keys per request.
Search frequency	50,000/hour ≈ 13.89 requests/second; repeated lookup performance dominates.
Update frequency	Daily additions allow index building or batch updating once per day rather than per query.
Linear search	Worst-case 50 billion comparisons/hour; not recommended.
Binary search	About 20 levels per request, around one million key comparisons/hour; predictable but requires sorted data.
Hash index	Expected $\Theta(n)$ build and $\Theta(1)$ per query; total expected $\Theta(n+q)$, with extra memory and collision considerations.

The evidence supports redesigning the retrieval layer around a hash index. This decision is not based only on Big-O: it accounts for the high number of repeated searches, the low update rate, the amortized preprocessing cost and the experimental separation between index-build and lookup time. A database-native index should be benchmarked before deployment.

14. Conclusion

Three different search strategies were designed, mathematically analysed and tested. Linear search is simple but scales poorly for large datasets. Recursive binary search has $\Theta(\log n)$ search time but requires sorted data. Hash-table lookup provides expected $\Theta(1)$ search time after expected $\Theta(n)$ preprocessing, using additional memory. The measured comparison counts matched these theoretical trends. Under the changed requirement of one million records, 50,000 searches per hour and daily updates, a hash index is the recommended design because its build cost can be amortized across a large number of queries.

Appendix A – Complete Python Implementation

Copy the following code into a file named student_search.py and run it with Python 3. The script includes the three algorithms and a basic experiment driver.

```
import time
import random

def linear_search(records, target):
    comparisons = 0
    for i, reg_no in enumerate(records):
        comparisons += 1
        if reg_no == target:
            return i, comparisons
    return -1, comparisons

def binary_search_recursive(records, target, low=0, high=None,
counter=None):
    if high is None:
        high = len(records) - 1
    if counter is None:
        counter = [0]
    if low > high:
        return -1, counter[0]
    mid = (low + high) // 2
    counter[0] += 1
    if records[mid] == target:
        return mid, counter[0]
    if target < records[mid]:
        return binary_search_recursive(records, target, low, mid - 1, counter)
    return binary_search_recursive(records, target, mid + 1, high, counter)
```

```
def build_hash_index(records):
    # Preprocessing: O(n) expected time and O(n) extra space
    return {reg_no: i for i, reg_no in enumerate(records)}

def hash_search(index, target):
    # Expected O(1) lookup
    return index.get(target, -1)

def run_experiment(n):
    records = list(range(1000000, 1000000 + n))
    target = records[-1]            # consistent worst case for linear search
    sorted_records = sorted(records)  # preprocessing for binary search
    start = time.perf_counter()
    idx_linear, linear_comparisons = linear_search(records, target)
    linear_seconds = time.perf_counter() - start

    start = time.perf_counter()
    idx_binary, binary_comparisons =
binary_search_recursive(sorted_records, target)
    binary_seconds = time.perf_counter() - start

    start = time.perf_counter()
    hash_index = build_hash_index(records)
    hash_build_seconds = time.perf_counter() - start
    start = time.perf_counter()
    idx_hash = hash_search(hash_index, target)
    hash_seconds = time.perf_counter() - start

    return {
        "n": n, "linear_idx": idx_linear, "linear_comparisons":
linear_comparisons,
```

```
        "linear_seconds": linear_seconds, "binary_idx": idx_binary,
        "binary_comparisons": binary_comparisons, "binary_seconds":
    binary_seconds,
        "hash_idx": idx_hash, "hash_build_seconds": hash_build_seconds,
        "hash_seconds": hash_seconds
    }
```

Appendix B – Reproducibility Notes

The timing table in this document was measured during document generation in the available Python runtime. Re-run the program on the target machine for a formal lab submission. For more reliable timings, execute each test at least five times and report the median. To test average behaviour, use random target positions; to test unsuccessful searches, use a registration number outside the dataset range. Do not present one machine's measured milliseconds as universal performance guarantees.